SOFTWARE ENGINEERING

Software Construction & Development

Lecture 1 • Introduction & Core Concepts

WHAT WE'LL COVER

Agenda

01 Introduction & Importance

02 Goals & Key Components

03 Core Concepts (Abstraction, Modularity...)

04 Lifecycle of Construction

05 Challenges & Guiding Principles

06 Key Activities & Best Practices

07 Tools, Metrics & SDLC Context

08 Worked Example & Takeaways

GETTING STARTED

What Is Software Construction?

Software Construction is the process of creating working software through **coding, testing, debugging, and integration.** It is a core activity within the software development lifecycle (SDLC).

It's not just writing code — ***it's about writing high-quality, maintainable, and reliable code.***

IN SIMPLE TERMS

Code + Quality + Testing + Maintenance

= Software Construction

THE BIGGER PICTURE

Why Is Software Construction Important?

Software is useless unless built correctly. Construction **bridges the gap between design and the actual working product.**

- ✓ **The product works as intended**
- ✓ **Code is clean and efficient**
- ✓ **Future updates are easier**
- ✓ **Prevents software bugs and vulnerabilities**

ENSURES THAT...

Good construction is the bridge between a great design on paper and software people can actually rely on.

WHAT WE AIM FOR

Goals of Software Construction

Correctness

The code behaves as expected

Maintainability

Easy to read, update, and debug

Efficiency

Uses memory & CPU effectively

Robustness

Handles errors gracefully

Reusability

Code can be reused across projects

Scalability

Handles growth in users / data

BUILDING BLOCKS

Key Components

01 Coding
Writing code that performs tasks

02 Unit Testing
Testing individual functions or modules

03 Debugging
Finding and fixing errors

04 Integration
Making components work together

05 Error Handling
Dealing with unexpected inputs

06 Code Optimization
Improving performance & readability

07 Documentation
Writing helpful notes for developers

CORE IDEAS

Key Concepts in Software Construction

A **Abstraction**

Focus on what an object does, not how it does it.

e.g. using sort() without knowing its algorithm

B **Modularity**

Breaking software into manageable, separate modules.

Easier to build, test, and reuse

C **Encapsulation**

Hiding internal workings from the outside.

e.g. a class exposes only necessary methods

D **Separation of Concerns**

Each part handles a specific function.

UI, logic, and data stay decoupled

STEP BY STEP

Lifecycle of Software Construction

1 Understand Requirements → 2 Design the System → 3 Write Code → 4 Test Code

5 Debug Issues → 6 Review Code → 7 Refactor → 8 Integrate & Deploy

WHAT CAN GO WRONG

Common Challenges

Poor Code Quality

Messy or overly complex code

Lack of Testing

Leads to unexpected bugs

Hard-to-Maintain Code

No structure or comments

Changing Requirements

Clients often change what they want

Integration Problems

Modules don't work well together

Performance Bottlenecks

Slow code due to poor logic

GUIDING RULES

Core Principles of Software Construction

1 **Minimizing Complexity**

Keep code simple and readable. Use modularity — break programs into small, manageable pieces.

2 **Constructing for Verification**

Write code that can be tested easily. Use error checking to validate assumptions.

3 **Reuse**

Reuse existing libraries, frameworks, and components wherever possible.

PUTTING IT INTO PRACTICE

Key Activities I — Coding & Construction

1. Coding

Use a consistent coding style (indentation, naming) and follow language-specific best practices.

Example: *use clear names like username instead of usrnm, and organize logic so others can follow it later.*

2. Code Construction Techniques

Top-Down
Start from main components and build down

Bottom-Up
Start with low-level utilities and build up

TDD
Test-driven development — write tests before code

PUTTING IT INTO PRACTICE

Key Activities II — Testing, Debugging, Integration

3 Unit Testing

Write automated tests for small components. Use frameworks like JUnit (Java) or PyTest (Python).

4 Debugging

Use debuggers and logging. Know the difference between symptoms and root causes — e.g. a form submitting twice due to a bad event handler.

5 Integration

Combine units into a working system, then run integration testing to ensure components work together.

DO IT RIGHT

Best Practices



Version Control

Use Git or similar systems to manage changes over time.



Documentation

Write comments and external docs to help others — and future you.



Code Reviews

Have others review your code for quality and correctness.

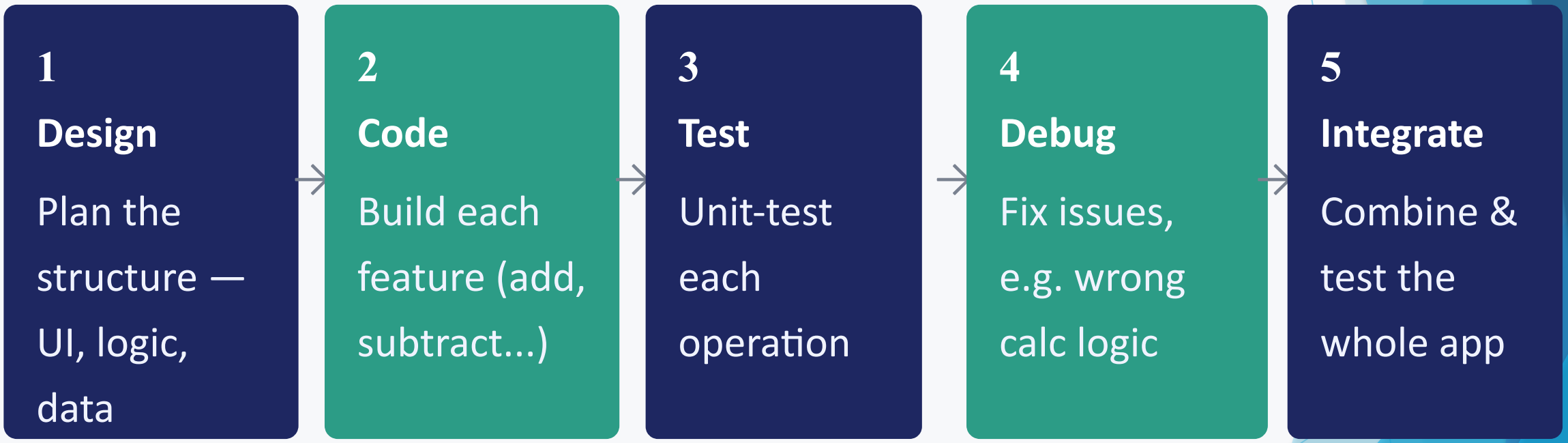


Continuous Integration

Automate building & testing for consistency and faster delivery.

SEEING IT IN ACTION

Example in Practice: Building a Calculator App



WORKING SMARTER

Tools of the Trade

Git / GitHub

Version control & collaboration

JUnit / PyTest

Automated unit testing

Docker

Consistent build & runtime environments

CI/CD (Jenkins, GitHub Actions)

Automated build, test & deploy

VS Code / IntelliJ

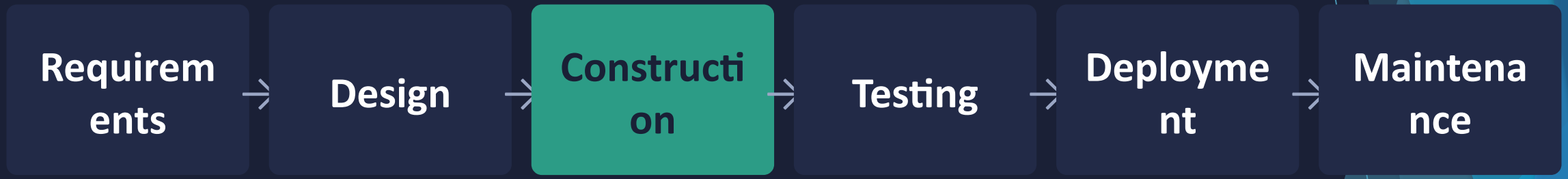
IDEs for writing & debugging code

Debuggers & Loggers

Diagnosing runtime issues

WHERE IT FITS

Software Construction in the SDLC



Construction sits at the center of the SDLC — it's where design becomes a working, testable product. Strong construction practices reduce rework in every phase that follows.

HOW DO WE KNOW IT'S GOOD?

Measuring Code Quality

80%+

Code Coverage

Share of code exercised by automated tests

< 10

Cyclomatic Complexity

Fewer independent paths = simpler, safer code

Low

Technical Debt

Shortcuts taken now that cost more to fix later

Low

Defect Density

Bugs found per unit of code — lower is better

QUICK REFERENCE

Do's and Don'ts

DO

- Write self-explanatory, well-named code
- Test as you build, not just at the end
- Refactor regularly to keep code clean
- Document decisions, not just syntax

DON'T

- Skip tests to save time
- Over-engineer for problems you don't have yet
- Ignore compiler / linter warnings
- Copy-paste code instead of reusing it

RECAP

Key Takeaways

1. Construction turns design into working, reliable software.
2. Correctness, maintainability, and robustness are non-negotiable goals.
3. Abstraction, modularity & encapsulation keep systems manageable.
4. Testing, debugging & integration are inseparable from coding itself.
5. Version control, reviews, and CI are the backbone of good practice.

THANK YOU

Questions?

Software Construction & Development — Lecture 1